

PROJECT REPORT

NetSage AI

An AI-Assisted Dashboard for Network Fault Diagnosis

Cisco–AICTE Virtual Internship 2026

Track: Applied AI + Network Troubleshooting

Submitted by

Shreesh Rathore

IILM University

August 2026

1. Introduction

As part of the Cisco–AICTE Virtual Internship 2026 under the Applied AI + Network Troubleshooting track, I set out to build NetSage AI — a tool concept aimed at helping junior network engineers move faster from "something is wrong" to "here is the actual root cause." This report documents what the internship problem statement asked for, what I was able to build within the timeline, and an honest account of how the two compare.

I've tried to keep this report grounded in what actually exists in the codebase today, rather than what the original plan described, because I think that distinction matters more for an evaluation than a polished-sounding summary would.

2. Problem Statement

Junior network engineers are usually comfortable running individual Cisco IOS commands, but they often struggle to connect an observed symptom — for example, "the PC gets an IP address but still can't reach the server" — to its actual root cause. That root cause could be sitting anywhere across the VLAN, routing, DHCP, DNS, ACL, NAT, or wireless layers, and figuring out which one takes experience that a junior engineer hasn't built up yet.

NetSage AI was conceived as a troubleshooting-assistant concept: you feed it a symptom, a short topology note, and some show-command output, and it returns a structured diagnosis. Because the output comes from an AI model, every diagnosis is meant to be reviewed by a human before anyone trusts it and acts on it — this human-in-the-loop requirement was a core part of the brief, in line with the program's Responsible AI expectations.

3. What I Actually Built

The delivered artifact (netsage-ai-insights.zip) is a single-page monitoring dashboard built with Lovable, designed to be screen-recorded for a short demo video. It's a self-contained React/TypeScript application that currently runs on mock, pre-written data rather than live AI calls.

This is narrower than the fuller architecture I originally planned, which included a live Claude API call, a CSV import pipeline, and a separate deterministic rule-checker module to cross-check the AI's diagnoses. Those pieces are documented as intended design in the project plan, but they are not present in this codebase yet — and I'd rather say that plainly than let the report imply otherwise.

Item	Detail
Frontend stack	React 18 + TypeScript, TanStack Start/Router, Vite, Tailwind CSS, shadcn/ui
Data source	12 hardcoded sample cases (src/lib/mock-cases.ts) — no live backend or database

Fault categories covered	VLAN (2), Routing (2), ACL (2), DNS (2), DHCP (1), Gateway (1), NAT (1), Wireless (1)
App sections (as built)	Monitoring Overview, Case Explorer, Responsible AI Log — 3 tabs
AI diagnosis engine	Not wired to a live API — each case carries a pre-written root cause, confidence, evidence, next command, and fix-steps field
Rule checker	Not present as a separate module in this build
Charts	Recharts bar chart (cases by issue type) and donut chart (Accepted / Edited / Rejected)

4. Features Implemented

4.1 Monitoring Overview (Dashboard)

- Summary cards for total cases, count by issue type, the AI-vs-human agreement rate, and the Rejected/Edited count.
- A bar chart showing cases broken down by issue type — VLAN, DHCP, DNS, Routing, ACL, NAT, Gateway, Wireless.
- A donut chart showing the split between Accepted, Edited, and Rejected review outcomes.

4.2 Case Explorer

- A sortable, filterable table of all 12 cases, showing symptom, fault type, OSI layer, confidence, and review status.
- Clicking a row opens a detail panel with the full AI diagnosis — root cause, evidence, next command, fix steps — shown side by side with the reviewer's verdict and note.

4.3 Responsible AI Log

- A filtered view of every case marked Edited or Rejected, each with a short reviewer explanation of why the AI's diagnosis was wrong or incomplete.
- This log is what satisfies the human-in-the-loop audit requirement — it's not enough to show the AI's answer, you also need a record of when and why a human overruled it.

5. How This Maps to the Original Deliverables

Planned deliverable	Status
Case dataset (30+ cases)	12 mock cases hardcoded — below the 30+ target
AI diagnosis output	Pre-written per-case fields; no live model call in this build
Dashboard	Implemented — Overview tab with summary cards and 2 charts

Responsible AI log	Implemented — dedicated tab with reviewer notes
Rule-based checker	Not implemented as a separate module
Demo video	To be recorded and shared via Google Drive

6. Limitations & Next Steps

- Case count (12) is below the 30+ required by the problem statement — I'd need to extend this either through a CSV import pipeline or by writing more hardcoded entries.
- No live AI call is wired up yet. The AI fields are static, so right now this build proves out the review workflow rather than live inference.
- There's no shared backend — everything lives in memory as mock data, so nothing persists between sessions or syncs across teammates.
- A deterministic rule-checker module, meant to independently cross-check the AI's diagnoses, doesn't exist in this repo yet.

If I were to continue this project, my priority order would be: wire up a live Claude API call, add the rule-checker as a sanity check against the model's output, push the case count past 30 (ideally via CSV import so it's not all hand-typed), and record the walkthrough demo.

7. Certifications Completed During the Internship

Alongside the project work, I completed three Cisco Networking Academy courses through IILM University, which fed directly into how I approached this project:

- Data Analytics Essentials — completed 11 June 2026
- Introduction to Modern AI — completed 11 June 2026
- Apply AI: Analyze Customer Reviews — completed 11 June 2026

These gave me the grounding I leaned on for structuring the mock case data, thinking about AI confidence scores, and framing the Responsible AI review workflow.

8. Conclusion

What I've delivered is a clean, demo-ready front end that proves out the core Responsible AI workflow the project set out to demonstrate: a monitoring dashboard, a browsable case table with the AI's diagnosis shown next to the reviewer's verdict, and a dedicated log of corrected or rejected cases.

It doesn't yet match the full scope of the original plan — the live API call, the rule-checker, and the larger case set are still ahead of me — but the workflow itself, and the reasoning behind it, is in place and working. The next steps are clear, and I've laid them out above rather than leaving them implied.